STATA Tutorial 3A

Index, Macros, and Loops

TIAN, Bing

March 11, 2025

Outline

Index

Macros

Loops
- forvalues
- foreach
- Applications

Index Data: _n, _N, [n], [n-k], [n+k]

```
. clear
. set obs 5
number of observations (_N) was 0, now 5
. gen X=_n*100
. dis X[1] " " X[2] " " X[3] " " X[4] " " X[5]
100 200 300 400 500
. dis _N
5
. replace X=X[_n+1]
(5 real changes made, 1 to missing)
. list X
```

	X
1.	200
2.	300
3.	400
4.	500
5.	.

```
. replace X=X[_n-1]
(4 real changes made, 4 to missing)
```

Outline

Index

Macros

Loops
- forvalues
- foreach
- Applications

Macros: “global” vs “local”

- The command global tells Stata to store everything in its memory until you exit Stata. If you open another data set before exiting, the global macro will still be in memory.
- The command local tells Stata to keep everything in memory only until the program or do-file ends.

```
. local city Beijing Shanghai Tianjin Chongqing
. global city Beijing Shanghai Tianjin Chongqing
. display "local: `city´"
local: Beijing Shanghai Tianjin Chongqing
. display "global: $city"
global: Beijing Shanghai Tianjin Chongqing
```

Micros: “local”

```
. *** local for Numeric Values
. local A 5
. display `A´
5
. local A=5
. display `A´
5
. local A 3+2
. display `A´
5
. local A=`A´+1
. display `A´
6
. *if we don´t assign values, by default it is 0
. local B=`B´+1
. display `B´
1
. *** local for Strings
. local C Beijing
. dis "`C´"
Beijing
```

Micros: “local”

```
. *** local with extended functions
. local n : word  count `city´
. dis `n´
0
. local one : word 1 of $city
. dis "`one´"
Beijing
. local l :  length local city
. dis `l´
0
. local pdir: sysdir PERSONAL
. dis "`pdir´"
/Users/bing/Library/Application Support/Stata/ado/personal/
```

Outline

Index

Macros

Loops
- forvalues
- foreach
- Applications

Loops: Syntax

Stata has three options for repeating commands over lists or values: **foreach**, **forvalues**, and **while**. Though each has a different first line, the syntax is consistent:

objects to repeat over

```
foreach x of varlist var1 var2 var3 {
```

open brace must appear on first line

temporary variable used only within the loop

requires local macro notation

```
    command `x', option
    ...
}
```

command(s) you want to repeat

can be one line or many

close brace must appear on final line by itself

stata cheatsheet

Outline

Index

Macros

Loops
- forvalues
- foreach
- Applications

Loops: “forvalues”

```
. forvalues i=1/4 {
  2.   dis "`i´"
  3. }
1
2
3
4
. forvalues i=4(-1)1 {
  2.   dis "`i´"
  3. }
4
3
2
1
. forvalues i=1(8)20 {
  2.   dis "`i´"
  3. }
1
9
17
```

Loops: "forvalues"

```
. set obs 5
number of observations (_N) was 5, now 5

. forval i=1/4 {
  2.   gen x`i´=`i´
  3.   label var x`i´ "variable `i´"
  4. }

. list in 1/2
```

	X	x1	x2	x3	x4
1.	.	1	2	3	4
2.	.	1	2	3	4

```
. des x1-x4

              storage   display    value
variable name   type    format     label      variable label
-------------------------------------------------------------------------------
x1              float   %9.0g                 variable 1
x2              float   %9.0g                 variable 2
x3              float   %9.0g                 variable 3
x4              float   %9.0g                 variable 4
```

Loops: “forvalues”

```
. local city Beijing Shanghai Tianjin Chongqing
. local code 010 021 022 023
. forvalues i=1/4 {
  2.   local name : word `i´ of `city´
  3.   local num  : word `i´ of `code´
  4.   dis "The area code of `name´ is: `num´"
  5. }
The area code of Beijing is: 010
The area code of Shanghai is: 021
The area code of Tianjin is: 022
The area code of Chongqing is: 023
```

Outline

Index

Macros

Loops
- forvalues
- foreach
- Applications

Loops: “foreach”

```
. foreach A of any a b c {
  2.          dis "`A´"
  3. }
a
b
c
. foreach A of local city {
  2.          dis "City: `A´"
  3. }
. foreach A of global city {
  2.          dis "City: `A´"
  3. }
City: Beijing
City: Shanghai
City: Tianjin
City: Chongqing
```

Loops: "foreach"

```
. foreach A of varlist x1-x4 {
  2.         sum `A´
  3. }
    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
          x1 |          5           1           0          1          1
    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
          x2 |          5           2           0          2          2
    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
          x3 |          5           3           0          3          3
    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
          x4 |          5           4           0          4          4
. foreach A in x1-x4 {
  2.         sum `A´
  3. }
    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
          x1 |          5           1           0          1          1
          x2 |          5           2           0          2          2
          x3 |          5           3           0          3          3
          x4 |          5           4           0          4          4
```

Nested Loops

```
. foreach var in X Y Z {
  2.         forval i=1/3 {
  3.                 gen `var´`i´=.
  4.         }
  5. }

. des, simple
X1  X2  X3  Y1  Y2  Y3  Z1  Z2  Z3
```

Outline

Index

Macros

Loops
- forvalues
- foreach
- Applications

Application 1: assign values to newly-generated variables

```
. * method 1
. foreach j of numlist 2 9 10 89 {
  2.     local i=`i´+1  /*same as "local ++i"*/
  3.     gen Avar`i´=`j´
  4.  }
. sum Avar*
```

Variable	Obs	Mean	Std. Dev.	Min	Max
Avar1	5	2	0	2	2
Avar2	5	9	0	9	9
Avar3	5	10	0	10	10
Avar4	5	89	0	89	89

Application 1: assign values to newly-generated variables

```
. * method 2
. local A 1 5 10 9
. forvalues i=1/4 {
  2.   local j: word `i´ of `A´
  3.   gen Bvar`i´=`j´
  4. }
. sum Bvar*
```

Variable	Obs	Mean	Std. Dev.	Min	Max
Bvar1	5	1	0	1	1
Bvar2	5	5	0	5	5
Bvar3	5	10	0	10	10
Bvar4	5	9	0	9	9

Application 1: assign values to newly-generated variables

```
. * method3
. local A xa xb xc xd
. local B 9 10 7 83
. forvalues i=1/4 {
  2.     local m: word `i´ of `A´
  3.     local n: word `i´ of `B´
  4.     gen `m´=`n´
  5. }
. sum x*
```

Variable	Obs	Mean	Std. Dev.	Min	Max
xa	5	9	0	9	9
xb	5	10	0	10	10
xc	5	7	0	7	7
xd	5	83	0	83	83

Application 2: process survey data quicker

```
. forvalues i=1/15 {
  2.     label var qb301_a_`i´ "Relation for sibling `i´"
  3. }

.
. foreach i of numlist 1 3/4 {
  2.     label var qb303b_a_`i´ "Age of sibling `i´"
  3. }

.
. foreach var in qb303y_a_1 qb303m_a_1 qb303b_a_1 {
  2.     sum `var´
  3. }
```

Variable	Obs	Mean	Std. Dev.	Min	Max
qb303y_a_1	3,360	700.6452	942.5526	-8	2001

Variable	Obs	Mean	Std. Dev.	Min	Max
qb303m_a_1	3,360	-3.672024	6.414588	-8	12

Variable	Obs	Mean	Std. Dev.	Min	Max
qb303b_a_1	3,360	35.68393	29.07707	-8	106

Application 2: process survey data quicker

```
. local B qb301_a_*
. *sum `B´
. *for varlist `B´: sum X
. foreach i of local B {
  2.     sum `i´
  3. }
```

Variable	Obs	Mean	Std. Dev.	Min	Max
qb301_a_1	3,360	.0675595	4.063417	-8	4
qb301_a_2	3,360	-1.151786	4.983348	-8	4
qb301_a_3	3,360	-2.773214	5.398201	-8	4
qb301_a_4	3,360	-4.434821	5.122894	-8	4
qb301_a_5	3,360	-5.955655	4.271327	-8	4
qb301_a_6	3,360	-7.012202	3.156648	-8	4
qb301_a_7	3,360	-7.59256	2.100002	-8	4
qb301_a_8	3,360	-7.85506	1.263264	-8	4
qb301_a_9	3,360	-7.961905	.6398319	-8	4
qb301_a_10	3,360	-7.977976	.4850766	-8	4
qb301_a_11	3,360	-7.994048	.2439387	-8	2
qb301_a_12	3,360	-7.99375	.2587376	-8	4
qb301_a_13	3,360	-8	0	-8	-8
qb301_a_14	3,360	-8	0	-8	-8
qb301_a_15	3,360	-8	0	-8	-8